`jwselect`: Full User Guide

Version 0.5.0

Doosoo Kim (doosoonet@gmail.com)

26 October 2026

`jwselect` selects optional preperiod restrictions and estimates treatment effects by minimum distance (MD), using unpooled cohort–time estimates and their full joint covariance. It provides manual, cohort-wise or joint Wald–BIC, and focused selection, plus joint pretrend tests. The maintained baseline is chosen by the researcher; selection does not establish its validity.

Version 0.5.0 adds optional CS-DR and nonlinear DiD adapters. This full manual covers installation, the input routes, all public options, interpretation, numerical inference, and computation across Stata sessions. Sections 1–4 cover routine estimation. Sections 5–7 explain focused inference and divided computation; Sections 8–10 cover results, checks, and troubleshooting. Appendix A is an option reference. Run multiline examples together from Stata's Do-file Editor.

Contents

1 Installation and updating **3**

2 Inputs and basic workflow **4**

2.1 Directly after a supported linear `jwdid` fit 4

2.2 CS-DR through `csdid` 5

2.3 Nonlinear DiD through `jwdid` 5

2.4 Meaning of the CS-DR and nonlinear adjustments 6

2.5 The general coefficient-and-covariance interface 6

2.6 A self-contained matrix example 7

3 Baseline, anticipation, and target horizon **7**

4 Selection methods and candidate families **8**

4.1 Baseline and manual restrictions 9

4.2 How MD uses a restriction set 9

4.3 Wald–BIC: cohort-wise or joint 9

4.4 Joint pretrend testing 10

4.5 Nested sets and all subsets 11

4.6 Focused point estimation and aggregation roles 11

4.7 Fixed aggregation weights . . . 12
4.8 Custom candidate families . . . 12

5 Inference and its numerical controls 13
5.1 Methods . . . 13
5.2 What numerical reselection does . . . 13
5.3 Defaults and interpretation . . . 14
5.4 Checking numerical stability . . . 14

6 Efficient serial computation 15
6.1 Compute inference only for needed outputs . . . 15
6.2 Memory and progress . . . 15

7 Divide a target across sessions 16
7.1 Prepare once . . . 16
7.2 Run a fixed worker partition . . . 16
7.3 Filters for a selected output, objective, or start . . . 17
7.4 Collect after completion . . . 18
7.5 What parallel execution preserves . . . 18

8 Stored results and diagnostics 19
8.1 Numerical diagnostics . . . 20

9 Package checks and reproducibility 21

10 Troubleshooting and common questions 22

A Command option reference 24

B Reference and contact 25

1 Installation and updating

The main estimation command requires Stata 16.1 or later. Preparation, workers, collection, and the release-test driver require Stata 17 or later. Extract the current distribution or GitHub source archive. The installation path is the directory containing `jwselect.pkg`, not the ZIP file or its enclosing directory:

```
net install jwselect, from("C:/path/to/jwselect") replace
which jwselect
which jwselect_worker
which jwselect_collect
findfile _jwselect_mata.do
help jwselect
```

Save your work and restart Stata after replacing an older release. Mata functions already loaded in a session do not automatically change when files on disk are replaced. Every worker must use the same release as preparation and collection. Keep a production run's executable files unchanged until it finishes; install a new release in a separate extracted directory/session.

The matrix interface requires neither upstream package. For the direct linear or nonlinear adapter, install `jwdid` and its dependencies separately:

```
ssc install require, replace
ssc install ftools, replace
ssc install reghdfe, replace
ssc install hdfe, replace
net install jwdid, ///
    from("https://raw.githubusercontent.com/friosavila/stpackages/main") replace
```

These upstream dependencies are separate from `jwselect`. Its adapters support narrower sets of models than the upstream commands themselves. See Section 2 for its restrictions. Upstream installation and covariance sources are listed at the end of this guide.

The CS-DR adapter targets `csdid` 2.0.1 and accepts its supported 2.x API from version 2.0.1 onward. This pins the source used for this release:

```
net install csdid, ///
    from("https://raw.githubusercontent.com/pedrohcgs/csdid-stata/
    f98bcea0691b1670c409bb7e3cc81d60705b4f84") replace
csdid version
```

For an existing installation, follow the authors' installation guide first to avoid conflicting registrations. Record the complete URL and reported version. The older SSC 1.x interface is unsupported; version 2.x needs no `drdid`. No `jwselect` command installs or updates an upstream package.

If your archive includes the `tests` directory, run short installation checks from that package root:

```
cd "C:/path/to/jwselect"
do tests/smoke.do
do tests/reselect_smoke.do
```

For the complete version-0.5 integration gate in a fresh Stata 17 session, run `do` with `run_release_`

tests.do from the package root. Its final success message ends with release tests: PASS. Section 9 describes a separate concurrent-session check. Tests replace data in memory; save your work first. Deliberately rejected invalid input is part of some tests: use the final test status to distinguish an expected rejection from an unexpected failure.

2 Inputs and basic workflow

2.1 Directly after a supported linear jwdid fit

```
jwdid y x1 x2, ivar(id) tvar(year) gvar(first_treat) ///
    never method(regress) cluster(id)
estimates store JW_SOURCE
jwselect, select(wbic) candidates(all) postwindow(0 2)
estat design
estat selection
estimates store JW_WBIC
```

Replace the variable names with yours. This example requires event times 0, 1, and 2 for every cohort. It explicitly uses all subsets; the command default is candidates(nested). Code never-treated units as first_treat = 0. Before changing a selection option, restore the upstream fit:

```
estimates restore JW_SOURCE
jwselect, select(manual) postwindow(0 2)
estimates store JW_BASE
```

This second specification is baseline only. Restore JW_SOURCE before *each* new direct-input specification, including focused selection, a changed baseline, or a different horizon. After jwselect, a bare jwselect redisplays its results. It does not refit the original model. Successful estimation replaces e() in the usual Stata manner.

The linear adapter supports balanced, regularly spaced panels; actual never-treated controls coded zero; unweighted linear fits; full cohort–time heterogeneity; clustering at the individual identifier; and plain numeric covariates that are constant within individual. Full heterogeneity corresponds to hettype(timecohort). Generate dummies or transformations explicitly; the adapter does not accept factor-variable or time-series notation. Unsupported weights, time-varying covariates, nonlinear specifications outside the scoped support below, and other cluster designs require separately validated matrix inputs. In particular, a weighted application with time-varying controls requires validated matrix inputs; it is not accepted merely because its first stage used jwdid.

vce(unconditional) is the jwdid-input default. It requests the joint covariance from estat attgt with vce(unconditional) and requires the upstream method(regress) fit. Explicit vce(conditional) uses the usual margins covariance and also supports the default reghdfe backend. The distinction concerns uncertainty in covariate averaging, not whether standard errors are clustered. Covariate averaging, where needed, occurs upstream; the MD stage uses the extracted estimates and covariance.

2.2 CS-DR through csdid

```
csdid y x1 x2, ivar(id) time(year) gvar(first_treat) ///
    method(dr) nevertreated base_period(universal) ///
    analytical pointwise cluster(id)
estimates store CS_SOURCE
jwselect, select(wbic) candidates(nested) postwindow(0 2)
estat selection
estat pretrend
estimates restore CS_SOURCE
jwselect, select(manual) postwindow(0 2)
```

The first CS-DR adapter requires unaggregated `method(dr)` results with never-treated controls and a universal base. It supports unweighted balanced panels with positive integer, unit-spaced calendar times and plain numeric covariates constant within individual. Clustering may be omitted or set to `ivar()`; panel influence functions already account for repeated unit observations. Higher-level clustering, unbalanced panels, repeated cross sections and time-varying covariates are outside this initial contract. Keep the original estimation data, and call the adapter before posting an aggregation. Missing cells, unsupported source metadata, bootstrap covariance, varying bases and not-yet-treated comparison groups cause errors.

`vce(supplied)` is the default and only CS-DR covariance convention. The adapter reads the full analytical influence-function covariance from the source, including cross-cohort and cross-period entries. It reconstructs structural-zero universal reference cells from upstream metadata and checks the mapping against the posted estimates. The source estimates are not recomputed at different bases.

Use `baseline(cohort)`. Omitted `anticipation()` is inferred from `csdid`; an explicit value must match. Existing linear and matrix rebasing behavior is unchanged. Restore `CS_SOURCE` before every new selection specification. See `examples/csdid_demo.do` for simulated data.

2.3 Nonlinear DiD through jwdid

```
jwdid y x1 x2, ivar(id) tvar(year) gvar(first_treat) ///
    never method(logit) cluster(id)
estimates store NL_SOURCE
jwselect, select(wbic) candidates(nested) postwindow(0 2)
estimates restore NL_SOURCE
jwselect, select(manual) postwindow(0 2)
```

The supported nonlinear methods are `logit` for a binary outcome and `poisson` for an exponential conditional mean. They retain the balanced, unweighted, never-treated, unit-clustered, full-cohort–time and plain-time-invariant-covariate contract above. Other nonlinear methods, including `ppmlhdfe`, are not supported by this initial adapter.

Joint `estat attgt` margins supply outcome-scale ATT and pre-treatment placebo estimates with their cross-covariances; nonlinear regression coefficients are not interpreted as outcome-scale ATTs. The default `vce(unconditional)` includes uncertainty from covariate averaging. `vce(conditional)` explicitly conditions on the observed covariate distribution. The adapter verifies the known zero-normalization rows, then sets numerical derivative residues no larger than 10^{-10} to exact zero. Materially nonzero baseline estimates or covariance entries cause an error.

baseline(cohort) is required. Omitted anticipation() is inferred as the recorded jwdid anticipation minus one. An explicit mismatch or baseline(common) is rejected. Restore NL_SOURCE before each new direct specification. See examples/nonlinear_demo.do for both methods.

2.4 Meaning of the CS-DR and nonlinear adjustments

For these two adapters, $\widehat{d}$ contains baseline outcome-scale ATTs and $\widehat{h}$ contains optional average outcome-scale placebo effects. For a fixed set A of zero-placebo restrictions, the covariance adjustment is

$$\widetilde{d}_A = \widehat{d} - \widehat{V}_{dh_A}\widehat{V}_{h_Ah_A}^{-1}\widehat{h}_A.$$

The covariance is the joint sampling covariance of these estimates. This is the canonical Gaussian MD problem with unrestricted post-treatment effects and $h_A = 0$. Its minimized quadratic statistic equals $\widehat{h}_A'\widehat{V}_{h_Ah_A}^{-1}\widehat{h}_A$. The equality is an identity for that compatible quadratic problem; it is not a claim that the procedure equals a constrained nonlinear refit or CS-DR re-estimation at alternative bases.

With covariate interactions, zero average nonlinear placebo effects can be weaker than setting every index-pretrend coefficient to zero. Identification, joint asymptotic normality, and consistent joint covariance remain necessary. CS-DR robustness conditions must apply to the baseline and each placebo moment used; the backend name alone does not establish those conditions. The MD adjustment can move nonlinear ATT estimates outside their natural bounds; results are not clipped. The fixed equalcohort/equalcell weights need not reproduce csdid's population-share aggregation. Selection and its inferential qualifications otherwise follow the common engine below. Upstream confidence intervals do not account for this selection step.

2.5 The general coefficient-and-covariance interface

```
jwselect, bmat(B) vmat(V) cells(C) nunits(500) ///
    gap(1) select(wbic) candidates(all)
```

Input	Required meaning
B	Row or column vector of m unaggregated cohort–time estimates.
V	$m \times m$ full sampling covariance of B, including cross-cohort and cross-time terms. Supply the covariance itself, not N times that covariance.
C	$m \times 2$ matrix of matching cohort and calendar-time rows, including normalization cells. Cohort codes are positive.
nunits()	Number of independent units, greater than one; used in the Wald–BIC penalty.
gap()	Time spacing; default 1.

The matrix route uses vce(supplied). The caller is responsible for the identification, target interpretation, normalization, and covariance of the supplied estimates. A full covariance is essential: treating cohort or time estimates as independent can change selection, MD weights, and inference. Already pooled estat event output generally lacks the cohort–time information needed for this calculation. Keep original normalization rows even when their estimates and covariance rows are zero. A normalization zero is not a precisely estimated effect. Rows may arrive in any order, provided B, V, and C match. Do not include never-treated rows in C; their comparison information is already reflected in the estimates and covariance. Missing values and duplicate cohort/date pairs are not allowed.

The raw covariance can be positive semidefinite, but each used restriction covariance must be nonsin-

gular. The command rejects invalid restrictions; it does not silently use a generalized inverse. Dates must align with `gap()`. Each included cohort needs its maintained baseline and at least one target postperiod.

2.6 A self-contained matrix example

This example needs no dataset or upstream estimator installation. It uses one cohort treated at date 4, dates 1–5, and a normalized baseline at date 3. The covariance is deliberately simple; retain the baseline row even though its estimate and covariance row are zero.

```
matrix B = (.05, .10, 0, .40, .60)
matrix V = .04*I(5)
matrix V[3,3] = 0
matrix C = (4,1 \ 4,2 \ 4,3 \ 4,4 \ 4,5)

* Baseline only; the cohort target averages dates 4 and 5.
jwselect, bmat(B) vmat(V) cells(C) nunits(500) ///
    select(manual)
estat design
matrix list e(hmap)

* Search all subsets of the two eligible dates, 1 and 2.
jwselect, bmat(B) vmat(V) cells(C) nunits(500) ///
    select(wbic) candidates(all)
estat selection
estat scores
```

The baseline-only target estimate is $(.40 + .60)/2 = .50$. The maps identify optional dates 1 and 2 in that order. Here post and precontrasts have zero covariance, so adding restrictions does not change the estimate or its fixed-selection variance; a general supplied covariance need not have this property.

To prespecify only the optional sets $\varnothing$, $\{1\}$, and $\{1, 2\}$, use the following local family. Column 1 is the actual cohort, followed by one bit for each row of the unchanged `e(hmap)`:

```
matrix CM = (4,0,0 \ 4,1,0 \ 4,1,1)
jwselect, bmat(B) vmat(V) cells(C) nunits(500) ///
    select(wbic) candmatrix(CM)
```

The family excludes $\{2\}$ alone. The empty local candidate is included once even if omitted from `CM`. In a multi-cohort input, each local mask has K bits for the full map, and a row may select only its own cohort's columns. Always construct masks for the baseline, anticipation, and input support used in the final call.

3 Baseline, anticipation, and target horizon

The syntax alternatives below are explanatory; enter one choice rather than the vertical bar.

```
jwselect, baseline(cohort|common) anticipation(#) ///
    postwindow(first last) select(manual|wbic|focused) ///
    candidates(nested|all)
```

Let g be a cohort's treatment date, q the earliest treated cohort in the input, h the time spacing, and δ the anticipation allowance.

Option	Meaning
`baseline(cohort)`	Baseline $g - h(1+\delta)$; default.
`baseline(common)`	Baseline $q - h(1+\delta)$ for every cohort.
`anticipation(#)`	Nonnegative integer δ; default 0 for linear/matrix inputs, inferred for the new adapters.
`postwindow(0 2)`	Event times 0, 1, and 2 in every cohort. Without this option, all available post-treatment cells are used.

The common-baseline and freely chosen anticipation rules in this section apply to linear and matrix inputs. The CS-DR and nonlinear adapters retain their upstream cohort baseline and infer anticipation if omitted.

Write the maintained baseline as b_g. For annual data with $\delta = 0$, these rules give $b_g = g - 1$ and $b_g = q - 1$. With $\delta = 1$, they are $g - 2$ and $q - 2$. The baseline is maintained as an identifying restriction and is never subject to selection. Optional dates satisfy $t < g - h\delta$ and exclude the baseline. Positive anticipation therefore removes potentially affected preperiods as well as moving the baseline. Post-treatment targets still start at treatment onset g.

With `baseline(common)`, eligible optional dates may be *later* than the baseline. For example, with $q = 4$, $g = 7$, $h = 1$, and $\delta = 0$, the common baseline is 3; observed dates 4, 5, and 6 are still eligible for cohort 7. The baseline need not be the last date in the selected subset. Although the theory allows an arbitrary maintained preperiod, this release exposes only the two rules above; it has no arbitrary baseline-date option.

A requested baseline and every requested post cell must exist. The command errors rather than silently changing the target horizon. The anticipation convention here differs from the upstream `jwdid` normalization option; do not mechanically pass the same numerical value to both commands.

Event time is $(t - g)/h$. Window endpoints must be nonnegative integers in increasing order (equal endpoints are allowed). Every intervening event time is included. Without a window, cohorts can have different exposure horizons. Each displayed cohort effect is the equal-weight mean of its included post cells. To estimate a single annual cohort effect rather than a multi-year average, use a singleton window such as `postwindow(0 0)`. This window applies to all cohorts in the input; requesting inference only for a particular output row is a separate choice.

4 Selection methods and candidate families

4.1 Baseline and manual restrictions

```
estimates restore JW_SOURCE
jwselect, select(manual)

matrix P = (4,1 \ 4,2 \ 6,3 \ 6,4)
estimates restore JW_SOURCE
jwselect, select(manual) prematrix(P)
```

Without `prematrix()`, manual selection uses only each maintained baseline. Matrix `P` has columns *cohort, calendar time*; list only optional dates, never the baseline. With default baselines for cohorts 4 and 6, the example adds dates 1,2 and 3,4 respectively. Dates need not be consecutive. This example assumes cohorts 4 and 6 and the listed dates are present; replace these with your own cohort/date pairs. A cohort absent from `P` uses its baseline alone. `candidates()` is inactive for manual selection.

4.2 How MD uses a restriction set

For linear/matrix inputs, let $\widehat{d}$ collect postperiod-minus-baseline contrasts and $\widehat{h}_A$ optional preperiod-minus-baseline contrasts. For the new adapters these are baseline ATTs and placebo moments, respectively. For a chosen set A, their sampling covariance blocks give the MD calculation

$$\widehat{d}_A = \widehat{d} - V_{dh_A} V_{h_A h_A}^{-1} \widehat{h}_A, \qquad V_{d,A} = V_{dd} - V_{dh_A} V_{h_A h_A}^{-1} V_{h_A d}.$$

For the empty optional set, the adjustment is zero. Cohort means and aggregates are linear combinations of these cell estimates. This covariance conditions on the chosen restrictions; data-dependent selection raises a separate inference problem.

4.3 Wald–BIC: cohort-wise or joint

`select(wbic)`, the default, minimizes

$$W_A - r_A \log N,$$

where W_A is the pretrend Wald statistic for candidate A, r_A is its number of optional restrictions, and N is the independent-unit count (`nunits()` for matrix input). The scope determines the candidate and covariance used in this criterion:

- `wbicscope(cohort)`, the unchanged default, selects one candidate within each cohort, using that cohort's restriction covariance. The final estimates use joint MD with the union of selected restrictions.
- `wbicscope(joint)` evaluates complete joint configurations with the full restriction covariance across cohorts and dates, selects one configuration, and uses it in joint MD for every output.

```
estimates restore JW_SOURCE
jwselect, select(wbic) wbicscope(joint) ///
    candidates(nested) postwindow(0 2)
estat selection
estat scores
estimates store JW_JOINT
```

Joint WBIC can choose a different set from cohort-wise WBIC, because joint Wald statistics generally do not equal the sum of cohort-specific statistics. In both modes, a restriction from another cohort can affect a final estimate through cross-cohort covariance. Separate cohort-by-cohort MD fits need not reproduce the joint fit. `wbicscope()` changes WBIC selection only; it does not change the focused FMSC criterion.

Under the compatible covariance-weighted MD model, W_A equals the corresponding minimized J statistic, including for joint restrictions. The package uses that identity; it does not independently refit a second model to compute J. Normal inference after either WBIC scope is a *pointwise oracle approximation* under its assumptions. It is not uniform inference under local violations. The selected-model Wald/J p-value remains a diagnostic and does not validate the maintained baseline.

4.4 Joint pretrend testing

After any successful fit, `estat pretrend` tests all eligible optional preperiod contrasts or the new adapters' outcome-scale placebo moments jointly. It uses the fitted design's baseline, anticipation cutoff, and full precontrast covariance; it does not restrict the test to the selection family or the selected set.

```
* Prespecified full optional set, jointly across cohorts
estat pretrend
matrix list r(table)

* Prespecified subset: replace these cohort/date pairs with yours
matrix Ptest = (4,1 \ 6,3)
estat pretrend, prematrix(Ptest)

* Selected mask for a manual or Wald-BIC fit
estat pretrend, selected
```

`prematrix(Ptest)` has two columns, cohort and calendar time. Every row must be an eligible optional date in `e(hmap)`; never list a maintained baseline. `selected` and `prematrix()` are mutually exclusive.

After focused selection, `selected` requires an explicit output row, because different targets can use different masks:

```
matrix list e(output_map)
estat pretrend, selected output(1)
```

For manual/WBIC fits, the output defaults to 1 because their joint mask is common to all outputs. `output()` is allowed only with `selected`; it specifies a row number, not a cohort date or event time. For the aggregate of cohort-focused estimates (`agg_cohort`), the selected mask is the union of cohort-selected restrictions. The reported J is the compatible joint-MD criterion for this tested set, not a residual criterion evaluated at the separately adjusted focused estimate.

For a nonempty test set A, the reported calculation is

$$W_A = \widehat{h}_A' V_{h_A h_A}^{-1} \widehat{h}_A, \qquad J_A = W_A, \qquad p_A = \Pr\{\chi^2_{r_A} \geq W_A\}.$$

$V_{h_A h_A}$ is the sampling covariance, not N times that covariance. The test checks that this restriction covariance is positive definite; it does not silently use a generalized inverse. The J value uses the covariance-compatible MD identity, not an independent refit or numerical verification of that identity. The compatible model leaves all postperiod effects unrestricted and uses the same covariance for Wald

and MD. Reporting J through the identity does not independently certify the full-covariance or linear-representation assumptions. For a fixed, prespecified set, the chi-square reference applies under the maintained model and regularity assumptions. For an outcome-selected set—including a custom set chosen after inspecting outcomes—the reference p-value is a diagnostic, not automatically a valid post-selection test. This command does not test the maintained baseline identifying assumption or replace the reported treatment-effect inference.

The command returns `r(chi2)`, `r(J)`, `r(df)`, `r(p)`, and `r(table)` with columns `df Wald J pvalue`. `r(test_set)` is `all`, `selected`, or `custom`; `r(postselection)` equals 1 for `selected`, otherwise 0. The latter is a record of the requested mode, not a certification that a custom set was prespecified. `r(output)` records the selected output, or zero for all/custom tests. `r(mask)` records the tested mask in `e(hmap)` order when $K > 0$; `r(preperiods)` lists tested cohort/date pairs when nonempty. With no restrictions, $r_A = 0$, $W_A = J_A = 0$, and the p-value is missing. The command leaves active estimation results, data, and RNG state unchanged.

4.5 Nested sets and all subsets

The baseline is always retained. If the optional dates are $\{1, 2, 3\}$:

Family	Optional sets searched
`candidates(nested)`	$\varnothing, \{3\}, \{2, 3\}, \{1, 2, 3\}$. Default: add dates from latest to earliest.
`candidates(all)`	$\varnothing, \{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}$.

With K optional dates there are $K + 1$ nested candidates and 2^K exhaustive candidates, including baseline only. The two families coincide for $K \leq 1$. All subsets allows gaps such as $\{1, 3\}$; a nested family does not. Eligibility depends on the anticipation cutoff, not on whether a date precedes a common baseline. For $g = 4$, common baseline 2, and no anticipation, the optional dates can be $\{1, 3\}$ and the nested sequence is $\varnothing, \{3\}, \{1, 3\}$.

`maxcand(4096)` is the default cap per local family; `maxjoint(4096)` caps a joint WBIC or aggregate-focused joint family. Exceeding a cap causes an error, not truncation. Raise a cap explicitly for larger prespecified families, subject to Stata's matrix limits and memory. For example, 14 optional dates imply 16,384 all-subset candidates and require at least `maxcand(16384)`. For joint WBIC and aggregate focus, local family sizes multiply: with K_g optional dates in cohort g, the Cartesian joint family has $\prod_g (K_g + 1)$ nested configurations or $2^{\sum_g K_g}$ all-subset configurations. For example, two cohorts with 7 optional dates each yield 16,384 joint all-subset configurations, despite only 128 candidates per cohort. The joint cap can bind even when every local family is below its own cap. Changing to a smaller family changes the statistical procedure; changing a cache budget or dividing identical searches does not.

4.6 Focused point estimation and aggregation roles

For baseline target variance v_0, candidate variance v_A, and candidate precontrast adjustment $a_A\widehat{h}$, the focused score on the sampling covariance scale is

$$Q_A = (a_A\widehat{h})^2 + 2v_A - v_0.$$

This uses an untruncated bias correction. The first stored candidate wins an exact tie. Generated families and custom local families put baseline only first. A supplied `jointmatrix()` retains the first-occurrence row order; put its empty row first to retain that tie convention. Focused selection is specific to a target; the best candidate can differ across annual effects of the same cohort. This covariance-based bias–variance criterion can be computed without invoking the Wald–J equality.

Option	Selection and reporting roles
`focus(cohort)`	Select each cohort's restrictions for its displayed post-period mean. Aggregate the resulting cell estimates with the chosen weights.
`focus(aggregate)`	Select a joint configuration for the aggregate; cohort estimates use that aggregate-selected model. They are not separately cohort-focused estimates.
`focus(both)`	Default: report cohort-focused effects, their aggregate, and a separately aggregate-focused estimate of the same aggregate target.

To get the focused point estimates without numerical inference:

```
estimates restore JW_SOURCE
jwselect, select(focused) focus(cohort) ///
    candidates(all) postwindow(0 0) inference(none)
```

This is useful for preparation and inspection. It does not supply selection-adjusted confidence intervals or p-values.

4.7 Fixed aggregation weights

`aggregate(equalcohort)`, the default, gives every cohort weight $1/G$ and divides its weight equally among included post cells. `aggregate(equalcell)` weights all included cells equally. Custom `aggweights(W)` overrides these defaults. `W` has columns *cohort, calendar time, weight*, includes each target cell exactly once (including zero weights), and has nonnegative weights summing to one.

```
matrix W = (4,4,1/6 \ 4,5,1/6 \ 4,6,1/6 \ ///
            6,6,1/6 \ 6,7,1/6 \ 6,8,1/6)
estimates restore JW_SOURCE
jwselect, select(wbic) postwindow(0 2) aggweights(W)
```

Weights are treated as fixed. Uncertainty from estimating population shares or other weights is not added. A custom cell-weighted aggregate need not be a weighted sum of the displayed uniform cohort means.

4.8 Custom candidate families

`candmatrix(CM)` supplies local families. Its first column is the actual cohort, followed by binary masks in `e(hmap)` order. Include rows for every cohort; each local candidate may select only its own cohort's precontrasts. With $K =$ `e(K)`, `CM` has $K + 1$ columns. The empty candidate is inserted first; duplicate nonempty candidates within a local family are rejected. `jointmatrix(JM)` supplies one full joint mask per row, with no cohort column. It is available for `select(wbic) wbicscope(joint)` and focused selection with `focus(aggregate)` or `focus(both)`, and replaces the Cartesian product of the local families. `JM` has K binary columns; the empty row is prepended only if absent, and duplicates are coalesced in first-occurrence order. Its rows need not be Cartesian-product members. Obtain the map from a baseline-only manual run and `estat design`; then restore the upstream fit or reuse the matrix input for final estimation. Families, targets, and weights should be specified before inspecting outcomes. Screening candidates with observed results and then treating the surviving family as prespecified changes the inference problem.

5 Inference and its numerical controls

5.1 Methods

Method	Interpretation
Manual	Fixed-restriction Gaussian inference.
Wald–BIC	Pointwise oracle Gaussian inference under the relevant assumptions.
Focused, `inference(auto)`	Resolves to numerical `inference(reselect)`.
Focused, `inference(envelope)`	Explicit legacy all-candidate envelope, generally more conservative.
`inference(none)`	Point estimates and diagnostic fixed-selection covariance; no intervals or p-values.

Focused results report `se_fixed`, a fixed-candidate diagnostic. It is not a selection-adjusted normal standard error. The command does not post conventional `e(V)` for focused selection. Use its reported intervals and p-values with the numerical qualifications below; do not reconstruct normal intervals from `se_fixed`.

5.2 What numerical reselection does

At a trial nuisance mean u for the optional precontrasts, simulate the joint Gaussian estimation error using the supplied covariance, rerun the actual focused rule for each draw, and compute the error of the selected estimator. The observed winning candidate is not held fixed. An aggregate of cohort-selected effects uses the same joint draw across cohorts, sums their selected errors, and then computes quantiles; it does not add separately optimized marginal interval endpoints.

The nuisance confidence region has the form

$$\mathcal{U} = \{u : (u - \widehat{h})' Q^{-1} (u - \widehat{h}) \le \chi^2_{k,1-\gamma}\},$$

using the active precontrasts, their dimension k, and covariance Q. The optimizer searches the equivalent standardized unit ball. For $\alpha = 1 - \texttt{level}/100$ and $\alpha_2 = \alpha - \gamma - \texttt{mcerr} > 0$, four objectives are:

1. Minimize the $\alpha_2/2$ selected-error quantile.
2. Maximize the $1 - \alpha_2/2$ selected-error quantile.
3. Maximize the selected-error left-tail probability at the observed estimate.
4. Maximize the selected-error right-tail probability at the observed estimate.

The last two searches are needed for the zero-null p-value: points with extreme quantiles need not maximize the relevant tail probabilities.

The native Mata optimizer is a multistart projected pattern search, not NOMAD. Each start has a fixed budget and evolves independently. Common search draws make comparisons across trial values less noisy. Independent validation draws subsequently check all four discovered points for all four quantities. Exact binomial order-statistic bounds and one-sided Clopper–Pearson bounds allocate `mcerr()/16` across the 16 finite-point bounds for each output. Let L be the minimum checked lower-error bound and U the maximum checked upper-error bound over all four points. Let p_L, p_R be the respective maxima of the checked tail-probability upper bounds. The report is

$$[\widehat{\theta} - U, \widehat{\theta} - L], \qquad p = \min\{1, \gamma + \texttt{mcerr} + 2\min(p_L, p_R)\}.$$

The independent simulation check controls error at the discovered points. **It does not control error from a more extreme, unvisited nuisance point.** Intervals are marginal, not simultaneous bands across outputs. Outputs with structurally zero reporting adjustments use ordinary normal inference without nuisance search.

5.3 Defaults and interpretation

Option	Default	Role
`optstarts()`	4	Starts for each objective
`optmaxeval()`	400	Total evaluations for each objective, across starts
`opttol()`	.02	Stopping mesh in standardized ball coordinates
`optreps()`	1,999	Common simulation draws during search
`reps()`	19,999	Independent validation draws
`seed()`	12,345	Reproducible simulation seed
`gamma()`	.01	Nuisance-region error allowance
`mcerr()`	.001	Finite-point simulation error allowance
`cachemb()`	64	Search-projection cache allowance in MB

For reselection, `optstarts()` must be at least 1, `optmaxeval()` at least the number of starts, `optreps()` at least 100, and $0 <$ `opttol` $< .5$. Validation draws must also be large enough for finite order-statistic bounds. If the command reports an infeasible simulation size, increase `reps()`.

The confidence level defaults to Stata's current level, usually 95 percent. Search budgets and simulation sizes should be chosen for numerical stability, not to obtain a desired significance result. More validation draws improve finite-point Monte Carlo accuracy; they do not expand the set of nuisance points searched. More starts, more search draws, or a larger evaluation budget address different aspects of numerical approximation.

A *budget-exhausted* status means a start reached its assigned limit before its mesh stopping condition. It is not a budget overrun or a Stata failure. The best point found is retained. Even a mesh-converged start does not certify a global extremum. `opttol()` is a search-coordinate tolerance, not a bound on confidence-interval endpoint error.

With `optstarts(8) optmaxeval(1600)`, each objective has a total budget of 1,600 evaluations, divided among eight starts (200 per start). It does not allocate 1,600 to each start. Dividing this work into eight sessions preserves that budget. More generally, each start receives the integer part of total budget divided by starts; the first starts receive one additional evaluation until the remainder is allocated. Raising the number of sessions alone does not strengthen the search.

Neither inference method promises a shorter interval than baseline. Smaller `gamma()` enlarges the nuisance region, so lowering it need not shorten intervals. The legacy DKW calculation belongs only to `inference(envelope)`, not to current reselection inference.

5.4 Checking numerical stability

Inspect `estat inference`. For an important or unstable target, compare the same design with more starts *and* a larger total budget. For example, compare `optstarts(4) optmaxeval(400)` with `optstarts(8) optmaxeval(1600)`. More starts with a fixed total budget leave fewer evaluations per start. Then consider more `optreps()` and `reps()`, smaller `opttol()`, and another seed.

Compare both endpoints, the p-value, objective values, and converged-start counts. A large validation

gap indicates simulation sensitivity at a found point; a budget flag calls for checking search effort. Neither measures the missed global extremum. Hold the target, candidate family, and statistical error allowances fixed when assessing numerical stability. Use a new prepared directory for each changed numerical configuration.

6 Efficient serial computation

6.1 Compute inference only for needed outputs

`inferoutputs()` specifies *output row numbers*, not cohort dates or event times. Inspect the map first:

```
estimates restore JW_SOURCE
jwselect, select(focused) focus(cohort) ///
    postwindow(0 0) inference(none)
matrix list e(output_map)

* Suppose row 1 is the cohort effect of interest.
estimates restore JW_SOURCE
jwselect, select(focused) focus(cohort) postwindow(0 0) ///
    inference(reselect) inferoutputs(1) cachemb(64) progress
```

The option suppresses costly inference for unrequested outputs, including an unused aggregate. It does not redefine the target, change the candidate family, or justify deleting other estimates or cross-covariance terms from the input. Unrequested rows retain point estimates and diagnostic standard errors, with inference absent. `nocohort` only changes display and should not be used as a substitute for controlling computation.

`focus(cohort)` is suitable when only cohort-focused targets are required. Using `focus(both)` additionally requests aggregate-focused point selection, which can itself require a large joint candidate family.

6.2 Memory and progress

`cachemb()` controls search-projection caching per active calculation; it is not a total Stata RAM limit. The allowance uses decimal MB. The engine caches as many initial candidate-projection columns as fit the allowance and streams the remaining candidates in their original order. There is no all-or-nothing cache threshold: for 9,999 search draws, 64 MB caches 800 double-precision candidate columns. `cachemb(0)` streams all candidates. Independent validation is processed in blocks rather than retaining the much larger complete validation-projection matrix. This numerical organization preserves the candidate choices and inference rule, apart from ordinary floating-point differences.

For illustration, $9{,}999 \times 16{,}384$ double-precision entries alone need about 1.31 GB (decimal units), excluding other arrays and overhead. A large cache per worker may be impractical when many workers run concurrently. Choose cache and worker counts jointly. More workers can lose performance if the operating system starts paging memory to disk.

`progress` reports starts and every 50 objective evaluations. A single evaluation can still take appreciable time for a large exhaustive family. Use it with visible command output; wrapping the command in `quietly` may suppress messages. Log silence alone is not an error. Neither a progress fraction nor completion of an early cohort gives a reliable overall runtime fraction when later cohorts have exponentially larger candidate families.

7 Divide a target across sessions

7.1 Prepare once

The parent of the new preparation directory must already exist. Preparation freezes the fitted numerical objects, candidate definitions, search and independent validation draws, starting points, and inferential settings. Use a *fresh* work directory for each design or numerical sensitivity configuration.

```
estimates restore JW_SOURCE
jwselect, select(focused) focus(cohort) ///
    candidates(all) postwindow(0 0) ///
    inference(none) inferoutputs(1) ///
    optstarts(8) optmaxeval(1600) optreps(9999) ///
    reps(49999) opttol(.01) seed(12345) ///
    cachemb(64) prepare("C:/work/target_run01") progress
```

Use `maxcand(16384)` when the intended local family requires that cap. Preparation performs point selection but does not run the expensive search. Save its estimation results to retain point estimates, selected restrictions, and the `e(hmap)` ordering needed to interpret collected nuisance means:

```
estimates save "C:/work/target_run01_estimates.ster", replace
```

This file is supplementary; worker and collection commands operate on the saved preparation and do not refit the raw-data regression.

An active output creates four objectives times the number of starts: with eight starts, 32 search jobs. An output requiring no nuisance search is handled analytically. Multiple selected outputs create their own job sets. Statistical correlation is retained in the common prepared joint covariance and draws; independent computation does not require independent moments or outcomes.

7.2 Run a fixed worker partition

Open four Stata sessions and use one command in each:

```
* Session 1
jwselect_worker using "C:/work/target_run01", ///
    worker(1) workers(4) cachemb(64) progress

* Session 2
jwselect_worker using "C:/work/target_run01", ///
    worker(2) workers(4) cachemb(64) progress

* Session 3
jwselect_worker using "C:/work/target_run01", ///
    worker(3) workers(4) cachemb(64) progress

* Session 4
jwselect_worker using "C:/work/target_run01", ///
    worker(4) workers(4) cachemb(64) progress
```

Job IDs are ordered by output, objective (1–4), then start. Partitioning uses the original job ID

even when filters are supplied. Jobs are assigned to workers; 32 jobs do not require 32 concurrent sessions. Every worker must access the same prepared directory. Completed jobs are saved separately. A completed-job marker is written only after its results are saved; repeated execution can reuse completed work. Configuration checks and per-job locks protect against incompatible or overlapping work. Locks are named `job_<id>.lock`; completion markers are named `job_<id>.bin.done`. Remove a stale lock only after establishing that its original worker process has stopped. Hotfix 6 retries removal of owned empty locks and reports persistent cleanup failures. Check `r(lock_failures)` after workers. Successful saved searches survive a cleanup failure, but collection still refuses remaining locks. Once all processes using the batch have stopped, remove only a confirmed stale empty lock directory, never the saved results or preparation directory. Do not edit prepared inputs or completed result files by hand.

For the standard partition, use the same `workers()` count and distinct indices. Alternatively, explicitly disjoint filtered job sets can coexist, including `worker(1) workers(1)` calls. To change a partition safely, first stop the old workers; compatible completed jobs can then be reused. A local disk is preferable for an active work directory to a cloud-synchronized folder whose synchronization may delay visibility of files. On an HPC system, choose a shared location visible to the jobs and observe the institution's Stata licence and scheduler requirements.

7.3 Filters for a selected output, objective, or start

The generic worker additionally accepts `outputs()`, `objectives()`, `starts()`, and `jobids()` as numeric lists. The last option lists global job IDs from that preparation. All filters are combined with AND and with the original-ID worker-partition condition. For example, a single session can process only objective 1 for output 1:

```
jwselect_worker using "C:/work/target_run01", ///
    worker(1) workers(1) outputs(1) objectives(1) ///
    cachemb(64) progress
```

To restrict that command further to start 3, add `starts(3)`. This selects jobs already frozen in preparation; it does not create a new start or change its allocated budget. The remaining jobs must also be completed before final collection. The generic worker has no `resume` or `redo` option; compatible completed jobs are reused automatically. The collector has no partial-results mode. These filters are examples, not a complete alternative schedule. Multiple assigned starts can be batched into one call, allowing reuse of the loaded context and projection cache:

```
jwselect_worker using "C:/work/target_run01", ///
    worker(1) workers(1) jobids(1 2 9 10) cachemb(64) progress
```

These example IDs refer to the original job index, not a renumbered subset. Repeated separate calls for individual starts can rebuild caches. Finer scheduling is therefore not automatically faster; account for preparation and memory overhead. A collection of disjoint filtered calls covering every required job is an alternative to the full worker partition; it computes the same estimators.

7.4 Collect after completion

```
jwselect_collect using "C:/work/target_run01"
matrix list r(table)
matrix list r(diagnostics)
matrix list r(extrema)
matrix list r(output_map)
matrix list r(jobs)
matrix list r(settings)
display r(N_jobs)
```

Collection currently performs validation serially; objective/start parallelization applies to the searches. The collector checks completeness and compatibility, combines starts using the objective's minimum or maximum and deterministic tie rule, and evaluates the four winning nuisance points using the prepared independent validation draws. It then forms the intervals, p-values, and diagnostics. An incomplete job set is not a completed inferential result. `r(N_jobs)` is the number of numerical jobs. If it is zero, `r(jobs)` is a one-row, four-column missing placeholder; the requested outputs are handled analytically.

Collection returns `r()` matrices and leaves active `e()` intact. It does not update `e(table)` or make `estat inference` display the collected intervals. In a fresh session, `estimates use` can reload the saved preparation estimates if their maps or selection details are needed; collection itself does not require that file. Copy `r()` results into named matrices or export them before a subsequent command overwrites `r()`. The generic collector does not automatically write CSV or Stata datasets. For example, copy results immediately:

```
matrix JW_TABLE = r(table)
matrix JW_DIAG = r(diagnostics)
matrix JW_EXTREMA = r(extrema)
matrix JW_MAP = r(output_map)
matrix JW_JOBS = r(jobs)
matrix JW_SETTINGS = r(settings)
scalar JW_NJOBS = r(N_jobs)

* Optional CSV export; preserve the data in memory.
preserve
clear
svmat double JW_TABLE, names(col)
generate long output = _n
order output
export delimited using "C:/work/target_run01/results.csv", replace
restore
```

Retain `JW_MAP` with the table to interpret its row numbers. Main-command point results can be saved from `e(table)` and `e(output_map)` similarly.

7.5 What parallel execution preserves

For a comparison with serial inference, use identical fitted inputs, targets, families, seed, error allowances, search and validation draws, starting points, budgets, tolerance, tie ordering, and cache allowance. Numerical equivalence is checked by the supplied tests with floating-point tolerances. Caching and streaming use the same formulas, but rounding near ties can alter a search trajectory; compare

their results using tolerances and search diagnostics. A restarted unfinished job may repeat its own unsaved search; completed jobs need not be rerun. Version 0.4.0 does not extract an in-memory search checkpoint from an older running command.

8 Stored results and diagnostics

Useful inspection commands after a main `jwselect` call are:

```
jwselect
jwselect, nocohort
jwselect, celleffects
estat design
estat selection
estat scores
estat pretrend
estat covariance
estat cells
estat inference
matrix list e(output_map)
ereturn list
```

The initial direct-input table uses calendar dates; omitted normalization rows are not precisely estimated zero effects. Final output rows are identified by `e(output_map)`, not by assuming a coefficient label contains a cohort number. In focused `focus(both)`, the aggregate of cohort-selected cells and the separately aggregate-selected estimate are distinct estimators of the same fixed aggregate target.

Let G denote the number of cohorts, P the number of included post cells, K the total number of eligible cohort–preperiod pairs, and O the number of reported targets. Usually $O = G+1$; focused `focus(both)` gives $G + 2$.

Stored matrix	Contents
`e(b)`, `e(table)`	$1 \times O$ estimates; $O \times 5$ table with columns `estimate`, `se_fixed`, `ll`, `ul`, `pvalue`.
`e(ci)`, `e(pvalue)`	$O \times 2$ reported interval endpoints and $O \times 1$ zero-null p-values. Missing where inference was not requested.
`e(V)`, `e(V_fixed)`	$O \times O$ joint covariance. `e(V)` is posted only for manual/WBIC; `e(V_fixed)` is always the fixed-selection covariance and is diagnostic under focused selection.
`e(output_map)`	$O \times 2$: output-kind code and actual cohort, with cohort zero for aggregates. Codes are listed below.
`e(baselines)`, `e(postmap)`, `e(hmap)`	Two-column maps: cohort and maintained baseline (G rows), target post date (P), or optional preperiod date (K).
`e(selection)`, `e(cell_selection)`	Binary masks for each reported target ($O \times K$) or cell ($P \times K$), in `e(hmap)` order. Records every restriction used, including other cohorts' restrictions in a joint fit.
`e(b_cells)`, `e(V_cells)`, `e(weights)`	$1 \times P$ selected cell estimates, $P \times P$ fixed-selection covariance, and $1 \times P$ aggregate weights, all in `e(postmap)` order. Cell SEs are diagnostics under focused selection.
`e(scores)`, `e(candidates)`	Candidate scores (columns below); local families with columns cohort, local candidate ID, and K mask bits.
`e(joint_candidates)`	Joint binary masks for joint WBIC or focused aggregate/both when $K > 0$.

Stored matrix	Contents
e(z), e(V_z)	Canonical $1 \times (P+K)$ contrast vector and joint covariance: post contrasts first, optional precontrasts second.
e(loading), e(targets)	$O \times (P+K)$ selected estimator loadings and fixed target maps.
e(inference_diagnostics), e(inference_extrema)	Numerical reselection diagnostics and discovered nuisance points, described below.
e(upstream_table)	Direct input only: source cohort, time, estimate, SE, statistic, p-value, lower and upper endpoint; precedes selection. CS-DR uses analytical pointwise inference.

e(output_map) codes are: 1, ordinary cohort row; 2, aggregate of cohort-focused cells; 3, aggregate-focused estimate; 4, manual/WBIC aggregate; 5, cohort under the aggregate-focused joint fit. Names such as cohort1 are row labels, not cohort dates. With $K = 0$, e(hmap), e(selection), e(cell_selection), and e(joint_candidates) are absent.

e(scores) columns are scope, cohort, candidate, nrestr, Wald, WBIC, estimate, se_fixed, adjustment, gain, and FMSC. Scope 1 is local; scope 2 is joint, with cohort zero; joint WBIC uses these scope-2 rows. WBIC minimizes the WBIC column; focused selection minimizes FMSC. The other columns are diagnostics. Local candidate estimates need not equal the final joint manual/WBIC estimate. No specification-test p-value is stored in this table; estat pretrend returns the requested test separately. Up to floating-point accuracy,

$$\texttt{e(loading)}\,\texttt{e(z)}' = \texttt{e(b)}', \qquad \texttt{e(loading)}\,\texttt{e(V_z)}\,\texttt{e(loading)}' = \texttt{e(V_fixed)}.$$

Scalars and recorded settings. Counts are N_units, N_observations, N_covariates, N_clusters, G, K, and P; unavailable matrix-input sample counts are missing. Scalars also record gap, anticipation, upstream_anticipation, level, gamma, mcerr, reps, seed, maxcand, maxjoint, cachemb, optstarts, optmaxeval, opttol, and optreps. Macros record baseline, selection_method, wbic_scope, focus, candidate_type, aggregate, covariance, inference, postwindow, sourcecmd, cmdline, inferoutputs, prepared_directory, and upstream_stat. Access each as e(name); ereturn list displays what is available. e(source_estimator), e(source_version), e(source_method), e(source_model), and e(restriction_scale) record the source and restriction convention. The source estimator is matrix, csdid, or jwdid. Source version is empty when not verified from source metadata, including jwdid; retain its which output separately. e(version) and e(build) identify the installed release.

8.1 Numerical diagnostics

There are four diagnostic rows for every reported output under reselection. The diagnostic columns describe:

- output, objective, nactive: output row, objective 1–4, and number of active nuisance coordinates.
- search_reps, validation_reps: draw counts.
- evaluations, starts_used, converged_starts: actual search effort and starts reaching the mesh criterion.
- budget_exhausted: Boolean flag (0/1); 1 means at least one start exhausted its budget, not a count of exhausted starts.

- `search_value`, `validated_value`, `validation_gap`: searched objective, independent empirical value at its discovered point before outward correction, and their absolute difference.
- `radius_fraction`: location of the winning standardized point; 0 is the center and 1 the boundary.

These are computational diagnostics, not specification tests. Independent validation differences diagnose simulation sensitivity at the located point, not distance from an unknown global extremum. The final interval or p-value may use a different objective's discovered point because every point is checked for all four quantities. Analytic outputs have zero search counts and missing objective values. Unrequested outputs retain four diagnostic rows with output/objective identifiers and missing remaining fields. To see the nuisance locations, use `matrix list e(inference_extrema)`; `estat inference` returns them but does not print the matrix.

9 Package checks and reproducibility

The release archive includes tests and examples, including the additional suites in `tests_extra/`. Current-release execution evidence belongs in `VALIDATION.md`; the following paragraph is the historical **version 0.4.0** record. Validation completed on 24 October 2026 used **Stata/MP 17 on 64-bit Windows**. Supplied logs confirm the earlier release-gate components, including 648 optimization-fit comparisons, the corrected joint-feature checks, 35 additional design checks, four saved-results scenarios, and 24 numerical-reference cases comprising 48 fits, 960 joint candidate comparisons, and 96 pretrend tests. The author also reported PASS for the separate `jwdid` integration suite and all 17 additional joint-selection cases after actual `jwdid` fits. Thus all four additional suites passed across the recorded runs. Production estimation code was unchanged during these additional tests.

These checks provide implementation evidence on the tested cases; they do not establish statistical coverage, global optimization, or runtime verification on every Stata version or platform. A fresh simultaneous four-session run and a clean-install check are not claimed by this record.

A sufficient staged validation workflow distinguishes four questions:

1. **Correctness on small examples.** Check baseline/manual/Wald–BIC estimates, selection scores, covariance, and focused point selection against reference calculations; run actual Stata public-command tests.
2. **Execution equivalence.** Compare serial and divided focused inference with frozen identical inputs. Compare caching with streaming and check that selecting one output leaves its results unchanged.
3. **Restart and collection.** Repeat completed workers; check rejection of missing jobs, incompatible inputs, and invalid filters. Confirm that collection has all required objectives and starts.
4. **Numerical adequacy and timing.** Benchmark an expensive empirical target, then compare endpoints and p-values under larger budgets, additional starts, more draws, and other seeds. This assesses stability, not a proof of global optimization.

The release includes `tests/joint040_reference.py`, an independent covariance-weighted, constrained minimum-distance calculation of the joint Wald/J identity and WBIC rule, and `tests/joint040_smoke.do`, which exercises the new Stata public API. In empirical use, the Wald statistic requires the appropriate fully robust joint covariance, including cross-cohort and cross-period terms; clustering must match the sampling design. The compatible MD criterion uses the same covariance construction. Synthetic covariance fixtures check the numerical identity, not the robustness of a covariance estimator. The upstream integration tests use the estimated joint covariance from supported unit-clustered source

workflows. The files tests/integration_csdid.do and tests/integration_nonlinear.do cover actual new-adapter fits when dependencies are available; tests/csdid_adapter.do supplies controlled CS-DR extraction fixtures. Test availability does not itself constitute execution or statistical-coverage evidence. The joint-feature smoke test is included in the automatic release gate:

```
cd "C:/path/to/jwselect"
do run_release_tests.do
```

Run the additional design, numerical-reference, upstream, and saved-results suites together with:

```
do run_extra_tests.do
```

Both drivers clear data and estimates; save your work first. The additional driver records each suite's result in summary.csv and its complete output in extra_tests.log within a new results directory under tests_extra/. It does not install or update upstream dependencies. The release gate exercises worker/collector APIs sequentially so one session can run the entire gate. Actual concurrent filesystem behavior has a separate check:

```
* Prepare once, using a new directory.
do tests/release030_concurrency.do prepare "C:/work/concurrency_test"

* Run one line in each of four separate sessions, from the package root.
do tests/release030_concurrency.do worker "C:/work/concurrency_test" 1 4
do tests/release030_concurrency.do worker "C:/work/concurrency_test" 2 4
do tests/release030_concurrency.do worker "C:/work/concurrency_test" 3 4
do tests/release030_concurrency.do worker "C:/work/concurrency_test" 4 4

* After all workers finish:
do tests/release030_concurrency.do collect "C:/work/concurrency_test"
```

Require the final message Actual concurrent worker results match serial: PASS. Retain the test and worker logs.

Begin with small test budgets. Move to an expensive target only after the command and divided-execution tests pass. Keep test results separate from final production results. Do not interpret a smoke test with deliberately small budgets as validating substantive inference.

Record the package build, Stata version, source fit and covariance convention, baseline and anticipation, target window, candidate family, weights, seeds, error allowances, numerical settings, and inference diagnostics. Separate application scripts can add their own extraction, scheduling, and CSV output; these are not additional generic jwselect options.

10 Troubleshooting and common questions

The command asks for a supported upstream fit. Restore JW_SOURCE before a new direct specification; active estimates may belong to jwselect. Alternatively supply bmat(), vmat(), cells(), and nunits() together.

Mata reports an old or incomplete build. Save your work and restart Stata after installing the complete package. Inspect which jwselect and findfile _jwselect_mata.do. If compilation fails, keep the first compiler error and loader path; later undefined-function errors may be consequences.

A baseline or postperiod is unavailable. Inspect input support and choose a supported design explicitly. The command does not silently shorten a horizon or substitute another baseline.

The candidate cap is exceeded. Count the intended local families and the joint product used by joint WBIC or aggregate-focused selection. Raise `maxcand()` or `maxjoint()` if memory permits. Choosing a smaller prespecified family changes the procedure. `inferoutputs()` does not avoid joint point selection requested by `focus(both)`.

A restriction covariance is singular. Check duplicated cells, normalization rows, covariance construction, and identification. Do not delete covariance terms or add arbitrary jitter merely to make a run complete.

Collection refuses to proceed. Finish every prepared job and wait for workers to exit. Inspect reported locks and compatibility messages. Do not mix preparations. There is no partial collection; separately prepared targets allow separate completion.

Intervals are missing or conventional postestimation fails. Check `inference(none)` and `inferoutputs()`. Focused estimates intentionally omit `e(V)`; `se_fixed` does not justify normal CIs. Do not use `e(V_fixed)` or `e(V_cells)` as selection-adjusted covariance for `lincom`, `test`, or `margins`. Observation-level `predict` is undefined. Use the provided intervals and p-values; rerun with another fixed target if needed.

Does the number of sessions change the estimator? No, provided the prepared inputs, jobs, and combination rule are the same. Changing the candidate family, target, baseline, or anticipation does change the statistical calculation.

Can the candidate subsets be split into independent confidence intervals? No. Reselection must find the winning candidate across the entire family for each draw. Separate subset intervals cannot simply be combined to reproduce that calculation. Objective/start splitting avoids this problem.

Why can a short yearly panel still be expensive? A cohort with K optional restrictions has 2^K candidates. Focused inference repeats selection across simulation draws and nuisance trials. MD point estimation from saved coefficients is much cheaper than this nested procedure.

Can increasing validation draws cure a budget warning? No. Validation and outer optimization address different numerical errors. Inspect start convergence and compare search settings separately.

Can I use the same directory for a different seed or larger budget? Use a new prepared directory. Results from different frozen designs must not be pooled as if they were one complete job set.

Does dropping an aggregate remove cross-cohort covariance? No. `inferoutputs()` controls which reported targets receive inference. The underlying joint covariance and the selection rule for the requested output remain intact.

How should I interpret an optional-restriction Wald/J diagnostic? Use `estat pretrend` for the prespecified full optional set, `estat pretrend, prematrix(P)` for a prespecified subset, or `estat pretrend, selected` for a selected-set diagnostic. Focused selected tests also require `output(#)`. The command reports equal Wald and J values through the compatible MD identity. A rejection concerns optional restrictions under the maintained model; it does not test the baseline identifying assumption. Outcome-selected p-values are diagnostics, not selection-adjusted tests. Switching estimators after inspecting a p-value creates another selection rule whose inference needs separate justification.

A Command option reference

Defaults below apply to a fresh estimation call. Omitted matrix options use no supplied matrix; omitted `inferoutputs()` requests every output. A bare `jwselect` replays active `jwselect` results. To change the specification, restore the upstream fit or supply the matrix inputs again.

Option and default	Meaning or restriction
`bmat(B) vmat(V) cells(C)`	Supply all three together for matrix input. `B` is a row or column vector; `V` is its full sampling covariance; `C` contains corresponding cohort and calendar-time rows.
`nunits(N)`, `gap(1)`	Matrix route: independent-unit count $N > 1$ and positive period spacing. Direct input obtains both from the upstream fit.
`vce()`	`jwdid`: default `unconditional` with `regress`, `logit`, or `poisson`; explicit `conditional` also permits linear `reghdfe`. CS-DR and matrix input use `supplied`.
`baseline(cohort)`	Linear/matrix inputs also allow `common`. CS-DR and nonlinear inputs retain the upstream cohort baseline. No arbitrary numeric baseline is implemented.
`anticipation()`	Nonnegative integer allowance. Default 0 for linear/matrix input. CS-DR/nonlinear input infers it from the source and rejects explicit mismatches.
`postwindow(first last)`	Inclusive nonnegative integer event-time endpoints, with first $\leq$ last. Default: all available post cells. Every requested cell must exist in every cohort.
`select(wbic)`	Alternatives: `manual`, `wbic`, `focused`.
`wbicscope(cohort)`	WBIC only: `cohort` selects within each cohort; `joint` minimizes joint WBIC using the full restriction covariance. Both then use joint MD.
`focus(both)`	Focused selection only: `cohort`, `aggregate`, or `both`.
`candidates(nested)`	Automatic local family: `nested` or `all`. Overridden by `candmatrix()`; inactive for manual selection.
`prematrix(P)`	Manual only: two columns, cohort and optional calendar date. Omitted means baseline-only; do not list baselines.
`candmatrix(CM)`	Automatic selection only: cohort followed by K binary mask bits in `e(hmap)` order. Provide a local family for every cohort.
`jointmatrix(JM)`	Joint WBIC or focused aggregate/both: K binary columns, one joint configuration per row. Overrides the Cartesian joint family.
`maxcand(4096)`, `maxjoint(4096)`	Positive integer caps per local family and on the joint family. Exceeding a cap causes an error.
`aggregate(equalcohort)`	Alternatives: `equalcohort` or `equalcell`. Weights are fixed.
`aggweights(W)`	Overrides aggregation defaults: cohort, time, weight; every target cell exactly once, including zeros. Nonnegative weights must sum to one; no renormalization.
`inference(auto)`	Resolves to fixed-restriction normal inference for manual, pointwise oracle normal inference for WBIC, and `reselect` for focused. Alternatives: `none`, `reselect`, `envelope`; the last two require focused selection.
`level(#)`	Stata's current confidence level, normally 95.
`gamma(.01)`, `mcerr(.001)`	Nuisance-region and simulation-error allowances. Their sum must leave positive error probability below $1 - \texttt{level}/100$.
`reps(19999)`, `seed(12345)`	Independent checking draws for reselection (simulation draws for envelope), and a nonnegative integer RNG seed. Too few draws for the requested bounds cause an error.
`optstarts(4)`, `optmaxeval(400)`	Starts and total evaluation budget *per objective*, shared across starts; budget must be at least the number of starts.

Option and default	Meaning or restriction
`opttol(.02)`, `optreps(1999)`	Unit-ball stopping mesh, strictly between 0 and .5; common search draws, at least 100.
`inferoutputs(numlist)`	Output row numbers, not dates. Focused with `none`, `auto`, or `reselect` only; other rows retain estimates but no intervals or p-values.
`prepare("newdir")`	Focused with `inference(none)` only. Requires a new directory with an existing parent; freezes a divided-work configuration without running searches.
`cachemb(64)`, `progress`	Nonnegative decimal-MB search-cache allowance per active calculation; zero streams all candidates. Not a total RAM limit. Progress messages are off by default.
`nocohort`, `celleffects`	Hide cohort result rows, or display cell estimates and fixed-selection SEs. Display only; these options do not reduce inference work.

Pretrend-test syntax

```
estat pretrend [, selected prematrix(P) output(#)]
```

The default tests all eligible optional restrictions jointly. `selected` and `prematrix()` are mutually exclusive. `output()` is allowed only with `selected`; it is required for focused selected tests and defaults to 1 for manual/WBIC selected tests. See Section 4.4 for returned results and interpretation.

Worker and collector syntax

```
jwselect_worker using "directory", worker(#) workers(#) ///
    [outputs(numlist) objectives(numlist) starts(numlist) ///
     jobids(numlist) cachemb(#) progress]

jwselect_collect using "directory" [, progress]
```

Brackets denote optional syntax; do not type them. All worker filters intersect the original-ID partition. The worker's own `cachemb()` default is 64; specify it explicitly to match preparation. The collector's `r(settings)` records preparation settings, not a worker override. Search settings cannot be changed by worker or collector options.

B Reference and contact

For the method, cite:

Kim, Doosoo. 2026. *From Pretrend Tests to Preperiod Selection in Difference-in-Differences*. Working paper.

Report the software version, including a hotfix number if applicable.

Questions and bug reports: doosoonet@gmail.com. Include your Stata version, command, first error, and a reproducible example.

Upstream documentation: `csdid` and `jwdid`. See Section 2 for the supported specifications.